DCTwin — Code Handoff Guide

A 5-page quickstart for taking over the codebase.

Condensed from `docs/DCTwin_User_Guide.md` (the full 16-page reference). Digital-twin dual-loop optimizer for weekly data-center cooling setpoints.

What DCTwin is

DCTwin is a digital-twin dual-loop optimizer for weekly data-center cooling setpoints. Each week it searches three global setpoints — CRAH supply-air temperature (SAT, 20–26 C), CRAH airflow (4.8–13.8 kg/s), and chilled-water supply temperature (CHWST, 13–19 C) — and broadcasts each candidate to 45 actuators (22 SAT + 22 FLOW + 1 CHWST, fixed order) for the controlled 1F-2A hall. There is no surrogate: every candidate is scored by a full-week EnergyPlus 9.5 run in Docker.

Two loops, one oracle

- Inner (per week): forecast (IT-load + weather) → beam search (coarse-to-fine) → EnergyPlus oracle → objective + feasibility → robust three-net safety gate → recommendation.json.
- Outer (deployment): pre-validate → expert approve → shadow deploy (writes 45 BMS commands, never actuates) → record realized KPIs → calibrate (bias/sigma + physics) → feed the next week's search.

HARD INVARIANT: rack inlet $\leq$ 26 C at every timestep, enforced in search (k.sigma pre-tighten), in the scenario ensemble, and as a 0-tolerance deploy backstop. Never weaken it to make a search succeed.

Repository layout

Package dctwin v3.0.3, Python $\geq$3.11,<3.15, managed with Poetry. Run Python from src/ (package roots are planner/, webapp/) or set PYTHONPATH=$DCTWIN/src. Most-used paths:

Path	Purpose
`src/planner/`	Inner loop: beam_search, oracle, objective, robust, forecaster, calibrator, broadcast, baseline, schedule. backtest_forecaster.py lives here.
`src/webapp/`	FastAPI backend: main.py (routes + SSE), jobs.py (JobRunner), store.py (SQLite), auth.py, status.py, telemetry.py, topology.py.
`src/frontend/`	React 19 + Vite + TS UI. pages/ (Dashboard, NewPlan, Review, Live, History, DigitalTwin3D), api.ts client. Node 22.
`src/*.py`	CLI entry scripts: plan_weekly.py, fit_forecaster.py, prevalidation.py, deploy.py, fit_recirc.py.
`dctwin/`	EnergyPlus/BCVTB co-sim engine: registration.py (make_env), gym_envs/ eplus_env.py, third_parties/docker_backend.py.
`src/runs/`	Per-week artifacts (gds-2024-MM-DD-HASH/): recommendation.json, realized.json, oracle/, robust/, prevalidation/, deploy/, plant/.
`models/ configs/ data/`	Planner assets, GITIGNORED. forecaster.pkl must be fit first. Copy GDS model from mycode/Tropical_DC_Files/GDS_Nov_Supply_Return32_CHWT_Backup.
`docs/superpowers/specs|plans/`	Dated spec+plan pairs per shipped tier; the design record.
`graphify-out/`	AST knowledge graph of src/ (GRAPH_REPORT.md, graph.html, graph.json) — navigation aid.
`scripts/clear-and-run.sh`	One-command launcher: clears state, builds frontend, serves whole app at http://localhost:8000 (–dev = Vite hot-reload at :5173).

Setup (first 30 minutes)

Goal: clone to a running env with a green test run. Pick the repo root once and reuse it everywhere; commands below are portable.

Convention — set these first (every later command uses them)

```
export DCTWIN=/path/to/your/dctwin/checkout
PY="$DCTWIN/.venv-dtwin/bin/python"  # or just `python` inside `poetry shell`
```

Prerequisites

Tool	Version	Needed for
Python	3.11–3.14	planner + webapp
Poetry	latest	Python deps (pyproject.toml + poetry.lock)
Docker	engine + EnergyPlus image	running a plan (oracle); NOT for unit tests
Node	≥18 (22 works)	frontend build/test

Pull the EnergyPlus 9.5 image (only needed to RUN a plan)

```
docker pull ghcr.io/cap-dcwiz/energyplus-9-5-0:latest
```

1. Install Python deps

poetry install creates the venv and installs package dctwin v3.0.3. On the reference machine the interpreter is the prebuilt $DCTWIN/.venv-dtwin/bin/python; on a fresh checkout it is whatever poetry provides (use ‘poetry shell‘ or ‘poetry run python‘).

```
env -C "$DCTWIN" poetry install
# then either: env -C "$DCTWIN" poetry shell   (gives you ‘python‘)
# or set: PY="$(env -C "$DCTWIN" poetry env info --path)/bin/python"
```

2. Copy the gitignored model assets

src/models, src/configs, src/data are gitignored and absent on a fresh clone. The planner cannot run without them. Copy the GDS model tree from the shared source (mycode/Tropical_DC_Files/GDS_Nov_Supply_Return32_CHWT_Backup — external to this repo; get it from a teammate or shared storage). The dirs must contain: models/forecaster.pkl, building.json, idf/; configs/dt (dt.prototxt) + policy; data/calibration.json, his_data_processed.csv, weather/ (EPW). forecaster.pkl must be fit before any plan.

Place assets under src/ (they live at src/models, src/configs, src/data — not repo root)

```
export GDS=/path/to/mycode/Tropical_DC_Files/GDS_Nov_Supply_Return32_CHWT_Backup
cp -r "$GDS/models" "$GDS/configs" "$GDS/data" "$DCTWIN/src/"
ls "$DCTWIN/src/models/forecaster.pkl" "$DCTWIN/src/data/weather"  # sanity check
# if forecaster.pkl is missing, fit it (run from src/ so ‘planner‘ imports resolve):
env -C "$DCTWIN/src" "$PY" fit_forecaster.py
```

3. Build the frontend

```
env -C "$DCTWIN/src/frontend" npm install
env -C "$DCTWIN/src/frontend" npm run build  # tsc -b && vite build → dist/
```

webapp/main.py mounts src/frontend/dist at / only when built; skip this and / shows a "not built" hint. scripts/clear-and-run.sh builds it for you.

4. Smoke test

Fast unit tests mock EnergyPlus via MockEvaluator (no Docker); pyproject addopts deselects integration tests. Run from src/ (package roots are planner/, webapp/) or set PYTHONPATH=$DCTWIN/src. Rely on exit code + the "N passed" count.

```
env -C "$DCTWIN/src" "$PY" -m pytest -q
# optional frontend tests (vitest):
env -C "$DCTWIN/src/frontend" npm test
```

Then launch the whole app at one origin (clears state, builds UI, serves backend at http://localhost:8000; –dev gives Vite hot-reload at :5173): scripts/clear-and-run.sh

Reference-environment note

- The reference sandbox strips a leading ‘cd‘, so the commands above use ‘env -C <dir>‘. On a normal shell you can just ‘cd‘ instead.
- On the reference sandbox Docker requires ‘sg docker -c "..."‘; e.g. integration tests: sg docker -c ’env -C $DCTWIN/src PYTHONPATH=$PWD $PY -m pytest -m integration -q’. On a normal machine drop the ‘sg docker -c‘ wrapper.

Running it end-to-end

Set the convention once; every command below uses it. PY is the project interpreter (a prebuilt venv on the reference machine; on a fresh checkout run poetry install, then PY is whatever poetry/venv provides, or just python inside poetry shell).

convention (paste once per shell)

```
export DCTWIN=/path/to/your/dctwin/checkout
PY="$DCTWIN/.venv-dtwin/bin/python"  # ref machine; fresh checkout: ‘poetry shell‘ -> PY=python
export PYTHONPATH="$DCTWIN/src"  # package roots are planner/, webapp/
```

Reference-environment note: the dev sandbox strips a leading cd (so it uses env -C <dir> ...) and needs Docker wrapped as sg docker -c "...". On a normal machine you can cd $DCTWIN/src and run docker directly; the commands below are written portably.

1. Fit the forecaster (once, required before any plan)

Pickles models/forecaster.pkl (room→column map + config). No plan runs without it. fit_forecaster.py:42 is a Python main(), not a CLI flag parser.

```
$PY $DCTWIN/src/fit_forecaster.py  # default his_data_processed.csv
# thread a weather file / method:
$PY -c "import fit_forecaster as f; f.main(weather_file='data/weather/<your-file>.epw')"
```

2. Launch the web app

clear-and-run.sh clears plan state, builds the frontend, and serves the whole app from one origin at http://localhost:8000 (UI at /, API at /api/*). –dev gives Vite hot-reload at :5173 proxying /api to :8000. The frontend MUST be built or / returns a "not built" hint instead of the UI (webapp/main.py mounts frontend/dist at / only when present).

```
$DCTWIN/scripts/clear-and-run.sh  # http://localhost:8000 (single origin)
$DCTWIN/scripts/clear-and-run.sh --dev  # http://localhost:5173 (hot reload)

# by hand (needs Docker to run real plans; build frontend first):
npm --prefix $DCTWIN/src/frontend run build
OPERATOR_TOKEN=op EXPERT_TOKEN=ex $PY -m uvicorn webapp.main:app --port 8000
```

3. Operator workflow (UI)

#	Step	Role	Endpoint
1	Paste token, log in	operator/expert	GET /api/plans (verify)
2	Create weekly plan (week_start, days, search params)	operator	POST /api/plans
3	Watch live search (level, evals, best score)	operator	SSE GET /api/plans/{id}/stream
4	Review setpoints, predicted KPIs, baseline, robust bands	expert	GET /api/plans/{id}
5	Approve (or reject)	expert	POST /api/plans/{id}/approve
6	Deploy approved plan (shadow mode)	expert	POST /api/plans/{id}/deploy
7	Monitor live racks / power / compliance	operator	GET /api/live, SSE /api/live/stream

4. Headless CLI flow

deploy.py exposes deploy(rec_path, oracle, ...) as a function (deploy.py:28), not a ___main___ CLI; it raises PermissionError unless status=="approved". prevalidation.py is advisory and carries the –approve/–reject flags.

```
$PY $DCTWIN/src/plan_weekly.py --week-start 2024-11-11 --days 7
$PY $DCTWIN/src/prevalidation.py --recommendation runs/<id>/recommendation.json  # validate (advisory)
$PY $DCTWIN/src/prevalidation.py --recommendation runs/<id>/recommendation.json --approve  # expert approve
# deploy (shadow): deploy(recommendation_path, ParallelEnvOracle(...), bms=ShadowBmsAdapter())
#   bms=None = pure-sim path (no shadow write). Requires status 'approved'.
```

5. Reading results

What	Where	Pass criterion
Predicted energy	recommendation.json → predicted_kpis	—
Realized energy	realized.json → total_hvac_energy_kwh	gap vs predicted = week error
Savings %	predicted_kpis.energy_reduction_vs_baseline_pct	(baseline-plan)/baseline x100
Safety: peak inlet	realized.json → inlet_temp_max_c	≤ 26.0 C
Safety: violations	realized.json → inlet_violation_steps	== 0 (else deploy_blocked)
Status	recommendation.json → status	pending_approval / approved / deployed; also blocked_unsafe, infeasible_fallback, deploy_blocked

Check recommendation.json schema is ≥ 1.7 before comparing baseline/savings fields (1.7 adds the as-operated baseline + energy_scope; 1.8 adds deploy_mode/bms/realized_source).

6. Tests

```
$PY -m pytest -q  # unit; EnergyPlus mocked (MockEvaluator), integration deselected
$PY -m pytest -m integration -q  # Docker-gated, real EnergyPlus (slow, BCVTB-flaky)
npm --prefix $DCTWIN/src/frontend test  # vitest
```

Run pytest from $DCTWIN/src (or with PYTHONPATH set). Rely on the exit code + the "N passed" count; the summary is often buried under warnings.

Architecture in brief

Inner planning loop (forecast → beam search → oracle → robust gate → recommendation)

run_weekly_plan (pipeline.py:62) builds a BeamPlanner and runs a best-first coarse-to-fine search over 3 globals: SAT [20,26 C], flow [4.8,13.8 kg/s], CHWST [13,19 C]. Level 0 scores a grid^3 coarse grid (default 5^3=125); each later level emits up to 8 local neighbors with a halving step. Every candidate gets one full-week EnergyPlus 9.5 run.

Objective (objective.py:47, minimized): score = energy_term + 1.0*inlet_excess_degc_steps + 0.2*rh_excursion_steps + 0.1*zone_temp_band_steps; +inf if not feasible. energy_term = total_hvac_energy_kwh, or weighted_energy_cost when a tariff is loaded (never both). The oracle computes the soft accumulators; the objective only sums them. Hard gate is_feasible (objective.py:35) is conjunctive: kpi.feasible AND inlet_violation_steps≤tol(0) AND inlet_temp_max+inlet_forecast_margin≤inlet_cap(26.0) AND (rh only if rh_hard).

Oracle: each candidate is scored by a real full-week EnergyPlus run in Docker (ghcr.io/cap-dcwiz/energyplus-9-5-0), fanned out across PROCESSES (the dctwin config is a process-global singleton) via ParallelEnvOracle. Candidate gen expands the 3-tuple to the GDS hall's 45 AGENT_CONTROLLED actuators in fixed order [22 SAT, 22 FLOW, 1 CHWST] (broadcast.py:59); a single SAT/FLOW value replicates, CHWST is global. This order is load-bearing — it must match dt.prototxt and bms.py. Running a plan needs Docker: on Linux bcvtb_host must be the Docker0 gateway 172.17.0.1 (OracleConfig default); non-Linux Docker Desktop needs host.docker.internal.

Three-net safety gate enforces inlet≤26 C with single-counted allocation (no double-hedging). Net 1 k.sigma pre-tighten: search cap = 26 - K_SIGMA*sigma (K_SIGMA=1.0; cold-start sigma=1.0 → cap 25.0). Net 2 robust ensemble: each finalist re-simulated across ~4 perturbed-plant scenarios, bias-corrected, tested vs the hard cap with margin cleared. Net 3 safety ladder: if all finalists fragile, enumerate the energy<→robustness frontier (CHWST down, then SAT down, then max-cooling corner) and pick the cheapest provably-robust variant; only the corner failing yields blocked_unsafe.

Outer deployment loop

Pre-validation (prevalidation.py:68) re-evaluates the recommendation with a fresh oracle (nominal + worst-case) right after planning, but is ADVISORY — it never fails the plan. Deploy (deploy.py:28) is approval-guarded; in shadow mode ShadowBmsAdapter.apply (bms.py:52) writes 45 denormalized commands to bms_commands.json but never actuates; realized KPIs still come from the oracle. Calibration (calibrator.py) fits per-KPI additive bias + sigma (fading floor) from paired deploy history; physics recalibration (recalibrator.py) maps a persistent energy bias to a fan-efficiency factor. As-operated baseline (baseline.py:41) derives current setpoints from telemetry medians, evaluated once per run so savings are honest.

Forecasters & webapp

Forecasters: IT-load persistence/seasonal (forecaster.py) — 1F-2A load is empirically flat, so week-to-week variation is weather-driven; weather is a historical-analog EPW spread (weather_forecast.py) generating nominal/hot/cool scenarios. Webapp: single-origin FastAPI backend (port 8000) serving a React 19 + Vite + TS frontend; single-worker JobRunner runs plans + deploys, SSE streams progress and live telemetry, fail-closed token auth (operator/expert).

Symbol	File:line	Role
`run_weekly_plan`	src/planner/pipeline.py:62	Inner-loop orchestration
`BeamPlanner.plan`	src/planner/beam_search.py:85	Coarse-to-fine search entry
`score / is_feasible`	src/planner/objective.py:47 / :35	Scalar objective + hard gate
`ParallelEnvOracle.evaluate`	src/planner/oracle.py:107	Process-parallel EnergyPlus scoring
`evaluate_one`	src/planner/oracle_worker.py:168	Per-worker full-week episode
`gds_action_spec`	src/planner/broadcast.py:59	45-actuator order [22 SAT,22 FLOW,1 CHWST]
`robust_select / safety_ladder`	src/planner/robust.py:132 / :64	Net-2 ensemble + net-3 backstop
`run_prevalidation_with_oracle`	src/prevalidation.py:68	Advisory replay gate
`deploy / ShadowBmsAdapter.apply`	src/deploy.py:28 / bms.py:52	Approval-guarded shadow deploy
`JobRunner / run_plan_job`	src/webapp/jobs.py:25 / :142	Threaded job queue + plan job

Invariants you must not break

- Inlet $\leq$ 26 C at every timestep — enforced in search (net 1), scenarios (net 2), and the deploy backstop (0-tolerance). Never relax it to make a search succeed.
- Process-based parallelism only — the dctwin/EnergyPlus config is a process-global singleton; never assume thread-safety for config-dependent state.
- No double-hedging — net-2 scenarios clear inlet_forecast_margin (robust.py); don't stack net-1's k.sigma inside net-2.
- Schema is versioned and ADDITIVE — recommendation.json 1.0 $\rightarrow$ 1.5 $\rightarrow$ 1.7 $\rightarrow$ 1.8; bump on shape changes and keep older readers working.
- Residuals fit against RAW predictions (residual_predicted_for) to prevent double-correction; the sigma fading floor never collapses to 0.
- Don't weaken tests — plans are test-first; implementers make code pass, never relax an assertion. Frontend build (tsc -b) type-checks tests with noUnusedLocals ON.
- Regenerate graphify-out/ after merging code changes (it's a navigation aid, not a build artifact).
- Don't push to origin without an explicit go-ahead — main is intentionally local-only.

Where to go deeper

- Full guide: docs/DCTwin_User_Guide.md — operator quick-start, per-module deep dives, results, extension recipes, and a complete file:line reference map.
- docs/superpowers/specs/ + docs/superpowers/plans/ — dated spec+plan pairs (test-first) for every shipped tier; read before any large change.
- graphify-out/GRAPH_REPORT.md + graph.html — AST knowledge graph of src/; the fastest way to navigate by symbol.
- src/README.md — headless CLI flow (plan_weekly.py, fit_forecaster.py, prevalidation.py, deploy.py); src/webapp/README.md — backend run/build details.
- Reference map at the end of the full guide (docs/DCTwin_User_Guide.md) — concern $\rightarrow$ primary file:line for every subsystem.